Name :- Sana Munir Alam
Roll # :- 24K-0573
Class :- BCS-5E

Computer Network [Assignment-2] Date:

Task 1

Q1) Reliable Data Transfer is essential because the disaster-response network carries critical info (like casualty list, supply, request etc). Lost, corrupted or out of order packets could cause incorrect decision, delay rescue operation or put lives at risk. Therefore the transport layer must supply reliability itself using checksum to detect corruption, sequence number to detect loss, duplicates and reordering, and ACK's with timeout retransmission to recover.

Q2) Stop-and-wait sends one packet and then waits a full RTT of 500ms for it's ACK, while sending the packet takes only 4ms.

Utilization $\Rightarrow U = \frac{L/R}{RTT + L/R} =$

where $\frac{L}{R} = \frac{8000}{2000000} = 4ms$ so $U = \frac{4}{500+4} = 0.00794$

$U = 0.79\%$

The sender is idle about ~~0.79~~ $(100 - 0.79 = 99.21\%)$ of the time. This results in low link utilization and wastes the available 2 Mbps bandwith. as only (Throughput $= 0.00794 \times 2 \times 10^6 = 15873$ bits/s $\approx$ 15 kpps) 15 kpps is achieved on the link. Every loss also costs a whole timeout period with nothing being sent.

Q3) Pipelining is useful here as it allows the sender to transmit multiple packets without waiting for an ACK ~~ere~~ after every packet.
By keeping "N" unacknowledged packets in transit during the 500 ms RTT, it improves bandwidth utilization and increases throughput

eg $N=4$ so $U = \min\left(1, \frac{N(L/R)}{RTT + L/R}\right) = \left(1, \frac{4(4)}{504}\right)$

$= (1, 0.0317)$

Date:

so $U = 3.17\%$.

Here we can see that the sender can transmit 4 packets before needing to wait for acknowledgement (16 ms worth of data during a cycle of approx 504 ms). The utilization is roughly 4 times the stop and wait. (whose was 0.79%).

Also Throughput increases meaning the link is more utilized.

$$\text{Throughput} = U \times R$$
$$= 0.0317 \times 2 \times 10^6$$
$$= 63.492 \text{ Kbps} \quad \{\text{approx}$$

Protocols such as GBN, SR use sender and reciever windows to manage multiple packet while maintaining reliable, in-order delivery to application.

Q4) SR is more suitable

Reason :- The bandwidth is scarce and the link is lossy. Many packets are in flight at any moment cause of long RTT. On a timeout, GBN resends the whole window and each resent packet wastes 4 ms of scarce capacity and is exposed to loss/corruption again. While SR resends only the packet that is actually missing.

SR keeps out-of-order packets that have survived the 500ms trip in reciever buffer and ACK each one seperately. They are never sent twice, and a lost ACK cost at most one retransmission.

However SR does need more buffer and timers, but this is acceptable for this scenerio.

Date:

TASK 2

2A) N = 4 {window size

- Sender Window [base, base + N - 1]
- [0-3] means 0,1,2,3
- ESN {expected sequence number
- retx # {retransmission number

- 10 packets numbered (0-9) no wrap around.
- one packet sent at a time.
- only first transmission of pkt2 is lost.
- timeout for pkt 2 fires after the out of order packet and their ACK have been handled, and before pkt 2 is successfully recieved again.
- → {means packet in flight x {means packet lost

Go Back - N

Event	Sender	Packet	Reciever	ACK	Window (sender)
1	Send pkt 0, start timer	0 →	...	...	[0-3] base = 0 next = 1
2	Send pkt 1	1 →	...	...	[0-3] next = 2
3	Send pkt 2	2 x lost	never arrive	...	[0-3] next = 3
4	Send pkt 3	3 →	...	...	[0-3] next = 4 window full
5	-	0 arrives	inorder → deliver expected seq num 0→1	ACK 0	[0-3]
6	-	1 arrives	inorder → deliver ESN 1 → 2	ACK 1	[0-3]
7	Recieve ACK 0 (cummulative) restart timer base updates 0→1	-	-	ACK 0	[1-4]
8	Send pkt 4	4 →	...	...	[1-4] next = 5
9	Recieve ACK 1 : restart timer base 1→2	-	-	ACK 1	[2-5]
10	-	3 arrives	out of order → discard	dup ACK 1	[2-5]
11	Send pkt 5	5 →	...	...	[2-5] next = 6 window full
12	-	4 arrives	out of order discard	dup ACK 1	
13	Recieve dup ACK 1 (from pk 3): ignored			ACK 1	[2-5]
14	-	5 arrives	out of order discard	dup ACK 1	[2-5]
15	Recieve dup ACK 1 (from pkt 4): ignored	-	-	ACK 1	[2-5]
16	Recieve dup ACK 1 (from pkt 5): ignored	-	-	ACK 1	[2-5]
17	TIMEOUT for pkt 2, Go back resend 2,3,4,5	-	-	-	[2-5]

Date:_____

Event	Sender	Packet	Reciever	ACK	Window (sender)
18	Resend pk2 (retx #1)	2→	...	...	[2-5]
19	Resend pkt 3 (retx #2)	3→	...	...	[2-5]
20	–	2arrives	expected→deliver ESN 2→3	ACK2	
21	Resend pkt 4 (retx #3)	4→	...	...	[2-5]
22	–	3arrives	in order→deliver ESN 3→4	ACK 3	[2-5]
23	Resend pkt 5 (retx #4)	5→	...	...	[2-5]
24	Recieve ACK 2 : base 2→3 restart timer	–	–	ACK 2	[3-6]
25	–	4arrives	inorder→deliver ESN 4→5	ACK 4	[3-6]
26	Send pkt 6 (new)	6→	...	...	[3-6] next = 7 window full
27	Recieve ACK 3 : base 3→4	–	–	ACK 3	[4-7]
28	–	5arrives	inorder→deliver ESN 5→6	ACK 5	[4-7]
29	Send pkt 7	7→	...	...	[4-7] next = 8
30	Recieve ACK 4 : base 4→5	–	–	ACK 4	[5-8]
31	–	6 arrives	inorder→deliver ESN 6→7	ACK 6	[5-8]
32	Send pkt 8	8→	...	...	[5-8] next = 9
33	Receive ACK 5 : base 5→6	–	–	ACK 5	[6-9] ~~next~~
34	–	7arrives	inorder→deliver ESN 7→8	ACK 7	[6-9]
35	Send pkt 9 (last packet)	9→	...	...	[6-9] next = 10 however all packets sent
36	Recieve ACK 6 : base 6→	–	–	ACK 6	[7-9]
37	–	8arrives	inorder→deliver ESN 8→9	ACK 8	[7-9]
38	Recieve ACK 7 : base 7→8	–	–	ACK 7	[8-9]
39	–	9arrives	inorder→deliver ESN 9→10	ACK 9	[8-9]
40	Recieve ACK 8 : base 8→9	–	–	ACK 8	[9]
41	Recieve ACK 9 : base 9→10 stop timer	–	–	ACK 9	[] done
	All 10 packets delivered				

GBN totals :- 14 transmissions (10 original + 4 retransmission), 13 ACKs (where 10 are unique, 3 are duplicate ACK 1).

Q2 B) Assumption.

- Reciever Variable : rcv-base, starting at 0.
- Each packet has its own timer.
- ✓ } marks a packet the sender has marked ACKed.
- x } means packet lost.

Event	Sender	Packet	Reciever	ACK	Window (sender)	Window (Reciever)
1	Send pkt 0 (own timer)	0→	...	...	[0-3]	[0-3]
2	Send pkt 1	1→	...	...	[0-3]	[0-3]
3	Send pkt 2	2 x lost	never arrives	...	[0-3]	[0-3]
4	Send pkt 3	3→	...	...	[0-3] full	[0-3]
5	—	0 arrives	in-window, in order deliver, rcv-base 0→1	ACK 0	[0-3]	[1-4]
6	—	1 arrives	in-window, in order deliver; rcv-base 1→2	ACK 1	[0-3]	[2-5]
7	Recieve ACK 0 : smallest base 0→1	—	—	ACK 0	[1-4]	[2-5]
	mark 0 ✓					
8	Send pkt 4	4→	...	...		[2-5]
9	Recieve ACK 1 : mark 1 ✓,	—	—	ACK 1	[2-5]	[2-5]
	base 1→2					
10	—	3 arrives	in-window : out of order →BUFFER	ACK 3 (individual)	[2-5]	[2-5] {3} Buffered
11	Send pkt 5	5→	...	...	[2-5] full	[2-5]
12	—	4 arrives	in-window ; out of order →BUFFER	ACK 4	[2-5]	[2-5] {3,4} Buffered
13	Recieve ACK 3 : mark 3 ✓	—	—	ACK 3	[2-5] (3✓)	[2-5] {3,4} Buffered
	not the smallest unAcked (pkt 2) so base stays 2					
14	—	5 arrives	in-window : out of order →BUFFER	ACK 5	[2-5] (3✓)	[2-5] {3,4,5} Buffered
15	Recieve ACK 4 ; mark 4 ✓	—	—	ACK 4	[2-5] (3✓,4✓,5✓)	[2-5] {3,4,5} Buffered
	base remains 2					
16	Recieve ACK 5 : mark 5 ✓	—	—	ACK 5	[2-5] (3✓,4✓,5✓)	[2-5] {3,4,5}
	window full can't send pkt 6.					

Date:_____

Event	Sender	Packet	Reciever	ACK	Window (Sender)	Window (Reciever)
17	TIMEOUT of Pkt 2 own timer, resend	–	–	–	[2-5] (3✓, 4✓, 5✓)	[2-5] {3,4,5} Buffer
18	Resend pkt 2 (retx # 1)	2 →	...	...	[2-5]	[2-5]
19		2 arrives	In-order, deliver 2 then 3,4,5 from buffer (4 packet at once) rcv-base 2→6	Ack 2	[2-5]	[6-9]
20	Recieve ACK 2 : mark 2✓ smallest → base jumps (2→6) as 3,4,5 already ✓	–	–	Ack 2	[6-9]	[6-9]
21	send pkt 6	6 →	...	...	[6-9]	[6-9]
22	send pkt 7	7 →	...	...	[6-9]	[6-9]
23	–	6 arrives	in-order → deliver rcv-base 6→7	Ack 6	[6-9]	[7-9]
24	Send pkt 8	8 →	...	...	[6-9]	[7-9]
25	–	7 arrives	in-order → deliver rcv-base 7→8	Ack 7	[6-9]	[8-9]
26	Send pkt 9	9 →	...	...	[6-9]	[8-9]
27	Recieve ACK 6 : mark 6✓ base 6→7	–	–	Ack 6	[7-9]	[8-9]
28	–	8 arrives	in-order → deliver rcv-base 8→9	Ack 8	[7-9]	[9]
29	Recieve ACK 7 : ~~base~~ mark 7✓ base 7→8	–	–	Ack 7	[8-9]	[9]
30	–	9 arrives	in-order → deliver rcv-base 9→10	Ack 9	[8-9]	[]
31	Recieve ACK 8 : mark 8✓ base 8→9	–	–	Ack 8	[9]	[]
32	Recieve ACK 9 : base 9→10 (All 10 packets delivered in order)	–	–	Ack 9	[] done	[]

SR Totals :- 11 data transmission (10 original + 1 retransmission)
10 Acknowledgements

Date:

Final Table	GBN	SR
Packet retransmitted	4 (pkt 2,3,4,5)	1 (pkt 2)
Total data transmission (original + retransmitted)	14	11
ACK's send (original + duplicates)	13	10

They differ as GBN reciever has no buffer. It discards pk 3, 4, and 5 for they are out of order even though they arrived correctly. As GBN responds on cummulative ACK hence on timeout the sender can't know which packet arrived so it goes and resends everything onward from last acknowledgement which was 1 hence pk 2 ~~then~~ was sent then 3, 4, 5 also got retransmitted.

SR has reciever buffer and ACK's individually. It keeps 3, 4 and 5 and send their acknowledgement. The sender knows exactly which packet is missing and as each packet has it's own timer. Only pk 2 is resent and when it arrives the reciever delivers 2, 3, 4 and 5 in order at once.

TASK 4

Report:-

Intro and Approach:

After an earthquake, rescue teams talk to the emergency operations centre over a satellite link with R = 2 Mbps, RTT = 500ms and 1000 byte packets so $L/R = 4ms$. The link loses, corrupts and delay packets and ACKs, and one wrong or missing packet can send team to wrong place. so every packet must arrive correctly and in order. Hence the need for reliable service on an unreliable channel.

We start with rdt 3.0 where a checksum detects corruption, seq #, detect duplicates, ACKs confirm recieipt, and a timer with retransmission recover lost packets.

Stop-and-Wait is too slow here as calculated in task 1 $U = 0.79\%$ which gives about 16 Kbps on a 2 Mbps link. Hence we added pipelining with a window of N packet

Date:

which gives Go-Back N and Selective Repeat. With N=4 the U is 3.17%. Filling the link would need about 126 packet in flight so the window size is the main performance limit on this link.

Implentations

rdt.py is a discrete-event simulator, a priority queue holds packet arrivals, ACK arrivals and timer expiries, and separate sender and receiver classes react to them. For every packet and ACK it independently draws loss, corruption (one flipped bit) and extra delay (50-150ms). A log line is written for every send receive, ACK, loss, timeout and retransmission.

- Packets: header = kind, seq number (idx mod 2^k, so no. wrarp) and a 16-bit internet checksum. A packet or ACK that fails the checksum is discarded.
- GBN: The sender keeps base, nextseqnum, the window [base, base+N-1] and one timer for the oldest unACKed packet. The ACKs are cummulative. On timeout it resends every packet from base to nextseqnum-1. The reciever only keep expectedseqnum. It delivers and ACK in order packet, and discards out of order or corrupted ones and reACKs the last in order packet.
- SR: The sender has one timer per unACKed packet and resends only the timed out one. The reciever has it's own window [rcv_base, rcv_base+N-1]. It ACKs each correct packet individually, buffers out of order ones and delivers in order. For an already delivered packet it resends the ACK, which is needed when an ACK was lost.
- Window Rules: GBN needs $N \leq 2^k - 1$ and SR needs $N \leq 2^{k-1}$, otherwise and old retransmission looks like a new packet. N=4 with k=3 is exactly SR's limit. The program refuses larger windows unless (--force) is given.
- Verification: every run checks the delivered data byte for byte, and 23 self tests pass.
- Assumption: - One packet leaves every L/R = 4ms, one way delay is 250 ms (an undisturbed packet is ACKed afte 504 ms). ACK transmission time is ignored, and timer starts when a packet is handled to the link. The timeout is 850 ms, above the worst case RTT of 804 ms. With a 700 ms timeout, a delayed ACK causes a premature transmissio

Date:

and SR accepted its stale copy as new data.

Experiments

Both protocols were run with 200 packets, N=4, k=3 timeout 850ms and seed 573 (my roll #). The scenerios were A reliable, B loss 0.10, and C loss 0.05, corruption 0.05 and delay 0.10, applied to packets and ACKs. Throughput = correctly delivered bits / simulated time. All six runs delivered 200/200 packet correctly and in order. python3 experiments.py --seed 573 reproduces everything.

Results

Scenerio	Protocol	Transmitted	Retransmission	Delivered	Time	Throughput(kbps)	Timeouts
A: reliable	GBN	200	0	200/200	25.21	63.46	0
A: reliable	SR	200	0	200/200	25.21	63.46	0
B: loss 0.1	GBN	312	112	200/200	54.72	29.24	28
B: loss 0.1	SR	248	48	200/200	50.27	31.83	48
C: mixed	GBN	312	112	200/200	57.88	27.64	28
C: mixed	SR	242	42	200/200	48.54	32.96	42

For Graph look into zip file results/gbn_vs_sr.png. You will see this detailed table in results.md

Comparison

On a reliable channel the two protocols are identical (200 transmission, 63.46 kbps) because with N=4 both are limited by the window, not by loss recovery, this matches 4 x 8000bits / 0.504s = 63.5 kbps. When packets are lost, GBN's reciever discards out of order packets and send only cummulative ACKs, so each of it's 28 timeouts resent the whole window of 4 (28x4=112 retransmissions), while SR buffers out of order packets and ACKs each one, so each of its 48 timeouts resent exactly one packet (48 retransmissions). Under mixed errors SR needed 42 retransmissions agains GBN's 112 (62.5% fewer) and finished 9.3s sooner (32.96 vs

Date:

27.64 kbps). SR time saving is much smaller than it's retransmission saving (8.7%, vs 57% in B), because a retransmission costs only 4 ms of link time, while the 850 ms timeout the both protocols must wait for dominates the cost of a loss.

~~Conclusion~~ :- Robustness

These are single-seed runs. Averaged over seeds, 573-592 retransmissions were 88 vs 46 (B) and 146 vs 48 (C) for GBN vs SR. Throughput was 32.7 vs 32.8 kpbs in B (a tie) and 23.7 vs 32.2 kbps in C. So SR's retransmission advantage is robust, but it's throughput advantage is clear only when corruption and delay are present.

Conclusion

Both protocols delivered every packet correctly and in order in all test, and both capped at about 63.5 kbps (3.17% utilization) by the window of 4 on this 500 ms link. GBN is simple with one timer and no receiver buffer. SR needs buffering and per packet timers but avoids most useless retransmission, so it's the better choice for this error-prone bandwidth limited disaster-response link. A larger window and a timeout above the worst case RTT would improve it further.

TASK 5

Q1) Stop and Wait sends one packet and then the sender sits idle for a whole RTT waiting for the ACK. Pipelining sends several packets back to back during that wait. The sender therefore ~~be~~ busy for $N(L/R)$ in every cycle instead of L/R while the cycle length stays $RTT + L/R$

Given : $L = 8000$ bits $R = 2 \times 10^6$ bps $RTT = 0.5$

Transmission Rate = $L/R = 8000/2 \times 10^6 = 0.004$ s

Stop and Wait $\Rightarrow U = T_R/(RTT + T_R) = 0.004/(0.5 + 0.004) = 0.794\%$

Pipelined ($N = 4$) $\Rightarrow U = (N \cdot T_R)/(RTT + T_R) = 4(0.004)/(0.504) = 3.17\%$

Date:

Throughput (Stop and Wait) = $0.00794 \times 2 \times 10^6 = 15.87$ kbps

Throughput (Pipelining N=4) $0.0317 \times 2 \times 10^6 = 63.49$ kbps

N = 4 gives exactly 4 times the utilization. It reaches 100% when $N \geq 0.504/0.004 = 126$ packets

Q2a) Sequence Number $\Rightarrow 2^k = 2^3 = 8$

Therefor the seq number are 0, 1, 2, 3, 4, 5, 6, 7. After 7, the sequence number wraps around to 0.

b) Max Safe window $\Rightarrow N \leq 2^{k-1} = 2^{3-1} = 2^4$

~~For~~ the seq space must be at least 2N, and $8 > 2 \times 4$.

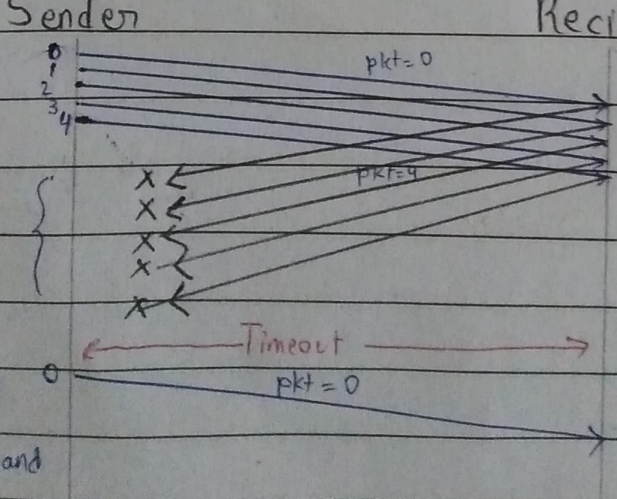
c) Large window causes ambiguity as the reciever can't see the sender's side. If the sender's old window and the reciever's new window overlap in sequency numbers, the reciever can't tell an old ~~transmission~~ retransmission from a new packet that happens to reuse the same number.

Diagram window N=5 (too big), seq number 0-7

seq numbers : 0 1 2 3 4 5 6 7 | 0 1 2 3 ...

Sender window : [0 1 2 3 4]

Reciever window : [0 1 2 3 4]



overlap at 0 and

1

Page Victory #8. Duplicate delivery

Date:

The reciever see exactly the same thing if the sender had really sent a new packet that reuses seq number 0. It can't distinguish the two cases, so the protocol fails.

Now if we had Diagram window. N=4 (safe)

